

Assessment 1 – Analytical Problem Solving

1. Water Jug Problem

Problem Statement

You are given two water jugs: one with a capacity of 4 gallons and another with a capacity of 3 gallons. Neither jug has measurement markings. The objective is to obtain exactly 2 gallons of water in the 4-gallon jug using only the available operations.

Assumptions

The pump can completely fill either jug.

Water can be poured from one jug to another.

Water can be emptied onto the ground at any time.

No additional measuring devices or containers are available.

State Representation

Each state is represented as (4-gallon jug, 3-gallon jug).

Initial State: (0, 0)

Goal State: (2, 0)

Solution Steps

1. Fill the 3-gallon jug. State becomes (0, 3).
2. Pour the water into the 4-gallon jug. State becomes (3, 0).
3. Fill the 3-gallon jug again. State becomes (3, 3).
4. Pour water from the 3-gallon jug into the 4-gallon jug until it is full. The state becomes (4, 2).
5. Empty the 4-gallon jug. State becomes (0, 2).
6. Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug. The final state is (2, 0).

Conclusion

The required goal is achieved because the 4-gallon jug contains exactly 2 gallons of water.

Model Used: State Space Model

Explanation:

The Water Jug Problem uses the **State Space Model** because each possible amount of water in the two jugs represents a state. The solution is found by moving from the initial state to the goal state using valid operations such as filling, emptying, and pouring water between the jugs.

```
from collections import deque

def water_jug():
    visited = set()
    queue = deque([(0, 0), []])

    while queue:
        (jug4, jug3), path = queue.popleft()

        if (jug4, jug3) in visited:
            continue
        visited.add((jug4, jug3))

        path = path + [(jug4, jug3)]

        if jug4 == 2:
            print("Solution:")
            for state in path:
                print(state)
            return

        next_states = [
            (4, jug3),          # Fill 4L
            (jug4, 3),          # Fill 3L
            (0, jug3),          # Empty 4L
            (jug4, 0),          # Empty 3L
            (jug4 - min(jug4, 3-jug3), jug3 + min(jug4, 3-jug3)), # 4→3
            (jug4 + min(jug3, 4-jug4), jug3 - min(jug3, 4-jug4)) # 3→4
        ]

        for state in next_states:
            if state not in visited:
                queue.append((state, path))

water_jug()
```

```
... Solution:
(0, 0)
(4, 0)
(1, 3)
(1, 0)
(0, 1)
(4, 1)
(2, 3)
```

2. Mars Rover as an Intelligent Agent

i. What are the percepts for this agent?

Percepts are the inputs received by the Mars Rover through its sensors. These include:

- Images captured by cameras.
- Rock and soil composition.
- Surface temperature.
- Terrain and obstacles.
- Atmospheric conditions.
- Battery level.
- Current location and direction.

These percepts help the rover understand its surroundings and make informed decisions.

ii. Characterize the operating environment

The Mars Rover operates in a challenging environment with the following characteristics:

Partially Observable: The rover cannot observe every part of the environment at once.

Dynamic: Weather conditions and terrain may change over time.

Sequential: Each action affects future decisions.

Stochastic: Some events, such as wheel slippage or dust storms, are unpredictable.

Continuous: Time, movement, and position change continuously.

Single Agent: The rover performs tasks independently without competing agents.

iii. What actions can the agent take?

- The Mars Rover can perform several actions, including:
- Moving in different directions.
- Capturing photographs.
- Collecting rock and soil samples.
- Drilling into rocks.

- Analyzing collected samples.
- Avoiding obstacles
- Sending scientific data back to Earth.
- Charging its batteries using solar panels.

iv. How can the performance of the agent be evaluated?

The rover's performance can be evaluated based on:
Number of successful scientific experiments.
Accuracy of collected data.
Safe navigation without damage.
Efficient battery and energy management.
Successful communication with Earth.
Completion of mission objectives within the planned time.

v. Which agent architecture is most suitable? Why?

A Utility-Based Agent is the most suitable architecture for the Mars Rover.

A utility-based agent evaluates different possible actions and selects the one that provides the highest overall benefit. It considers multiple factors such as battery level, scientific importance, safety, distance, and time. This allows the rover to make intelligent decisions even in uncertain situations.

```
from collections import deque

graph = {
    'Start': ['A', 'B'],
    'A': ['C'],
    'B': ['D'],
    'C': ['Sample'],
    'D': ['Sample'],
    'Sample': []
}

def bfs(start, goal):
    queue = deque([[start]])
    visited = []

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node == goal:
            print("Path to Sample:", " -> ".join(path))
            print("Sample Collected!")
            print("Mission Completed!")
            return

        if node not in visited:
            visited.append(node)

            for neighbor in graph[node]:
                new_path = list(path)
                new_path.append(neighbor)
                queue.append(new_path)

    bfs("Start", "Sample")

*** Path to Sample: Start -> A -> C -> Sample
Sample Collected!
Mission Completed!
```

Model Used: Goal-Based Agent

Explanation:

The Mars Rover is a **Goal-Based Agent** because it performs actions to achieve specific mission objectives, such as exploring the Martian surface, collecting samples, avoiding obstacles, and transmitting data back to Earth. Every decision is made to reach the predefined goal successfully.

3. 8-Queens Problem

Problem Formulation

The objective of the 8-Queens problem is to place eight queens on an 8×8 chessboard so that no queen attacks another queen.

Initial State

An empty chessboard with no queens placed.

State Space

All possible arrangements of eight queens on the chessboard.

Actions

Place one queen on the chessboard in a valid position.

Goal State

A valid arrangement where:

No two queens are in the same row.

No two queens are in the same column.

No two queens share the same diagonal.

Path Cost

Each queen placement has a cost of one step. The total cost is eight placements.

Search Strategy

The most suitable search strategies are:

Backtracking Search

Depth-First Search (DFS)

These methods efficiently eliminate invalid configurations and find a correct solution.

```
from collections import deque

N = 8

def is_safe(board, row, col):
    for r, c in board:
        if c == col or abs(r - row) == abs(c - col):
            return False
    return True

def bfs():
    queue = deque([[[]]]) # Empty board

    while queue:
        board = queue.popleft()
        row = len(board)

        if row == N:
            return board

        for col in range(N):
            if is_safe(board, row, col):
                queue.append(board + [(row, col)])

    solution = bfs()

    print("Queen Positions:")
    for queen in solution:
        print(queen)

... Queen Positions:
(0, 0)
(1, 4)
(2, 7)
(3, 5)
(4, 2)
(5, 6)
(6, 1)
(7, 3)
```

Model Used: Constraint Satisfaction Problem (CSP) Model

Explanation:

Backtracking or DFS

4. OLA Cab Booking Application

Type of Problem-Solving Agent

The most suitable agent is a Goal-Based Agent.

Explanation

The customer's goal is to travel from the source location to the destination. The agent evaluates the available cab options based on the customer's preferences, such as cab type, fare, waiting time, comfort, and availability. It then selects the option that best satisfies the user's goal.

Pseudocode

START

Input source location

Input destination

Input preferred cab type

Search available cabs

IF cab is available THEN

 Calculate fare

 Display estimated arrival time

 Show available cab details

 Confirm booking

ELSE

 Display "No cab available"

ENDIF

END


```
# Greedy Best-First Search for OLA Cab Selection

cabs = {
    "Mini": 150,
    "Micro": 120,
    "Sedan": 220,
    "Prime": 300,
    "Shared": 100
}

source = input("Enter Source: ")
destination = input("Enter Destination: ")

visited = []

while cabs:
    # Select the cab with minimum fare (Greedy choice)
    best = min(cabs, key=cabs.get)
    visited.append(best)

    print("\nChecking:", best)

    print("\nBest Cab Found!")
    print("Source:", source)
    print("Destination:", destination)
    print("Cab Type:", best)
    print("Fare: Rs.", cabs[best])
    print("Booking Confirmed!")
    break

... Enter Source: Thandalam
Enter Destination: Kilpauk

Checking: Shared

Best Cab Found!
Source: Thandalam
Destination: Kilpauk
Cab Type: Shared
Fare: Rs. 100
Booking Confirmed!
```

Model Used: Goal-Based Agent

Explanation:

The OLA Cab Booking application is a **Goal-Based Agent** because its primary objective is to help the user reach the desired destination. It selects the most suitable cab based on factors such as availability, fare, estimated arrival time, and customer preferences to achieve the user's goal.

5. Uniform Cost Search (UCS)

Problem Statement

A logistics company wants to transport packages from the starting warehouse S to the destination warehouse G with the minimum travel cost. Uniform Cost Search (UCS) is used to determine the least-cost path.

Edge Costs

$$S \rightarrow A = 1$$

$$S \rightarrow G = 12$$

$$A \rightarrow B = 3$$

$$A \rightarrow C = 1$$

$$B \rightarrow D = 3$$

$$C \rightarrow D = 1$$

$$C \rightarrow G = 2$$

$$D \rightarrow G = 3$$

Uniform Cost Search Steps

Step 1: Start from node S with a cost of 0.

Step 2: Expand node A because it has the lowest cost (1).

Step 3: From A, expand node C, which has the next lowest cumulative cost (2).

Step 4: From C, generate two possible paths:

To D with a total cost of 3.

To G with a total cost of 4.

Step 5: Expand D. The path through D to G has a total cost of 6, which is greater than the existing cost of 4, so it is ignored.

Step 6: Expand G with the lowest available cost. The goal is reached.

Optimal Path

$S \rightarrow A \rightarrow C \rightarrow G$

Total Cost

$S \rightarrow A = 1$

$A \rightarrow C = 1$

$C \rightarrow G = 2$

Total Cost = 4

Conclusion

Uniform Cost Search successfully finds the least-cost path from S to G. The optimal route is $S \rightarrow A \rightarrow C \rightarrow G$, with a minimum total cost of 4.

```

import heapq

graph = {
    'S': [('A', 1), ('G', 12)],
    'A': [('B', 3), ('C', 1)],
    'B': [('D', 3)],
    'C': [('D', 1), ('G', 2)],
    'D': [('G', 3)],
    'G': []
}

def ucs(start, goal):
    pq = [(0, start, [start])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node == goal:
            print("Shortest Path:", " -> ".join(path))
            print("Minimum Cost:", cost)
            return

        if node in visited:
            continue

        visited.add(node)

        for neighbor, weight in graph[node]:
            heapq.heappush(pq, (cost + weight, neighbor, path + [neighbor]))

ucs('S', 'G')

```

... Shortest Path: S -> A -> C -> G
Minimum Cost: 4

Model Used: Graph Search Model (Uniform Cost Search Algorithm)

Explanation:

Uniform Cost Search uses the **Graph Search Model** because the problem is represented as a graph consisting of nodes and weighted edges. The UCS algorithm systematically explores paths in order of increasing cost and guarantees finding the least-cost path from the start node to the goal node.